

AI ASSISTED CODING

NAME : P. Samanvith

BATCH : 40

ROLL NO : 2303A52090

Lab 15 – Backend API Development: Creating RESTful services with AI

Task Descriptor-1: (Setup Fast API Backend)

Instructions:

- Install FastAPI and Uvicorn.
- Use AI to generate a basic FastAPI application with one endpoint

Sample Code:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")

def read_root():

    return {"message": "Welcome to AI-powered FastAPI"}
```

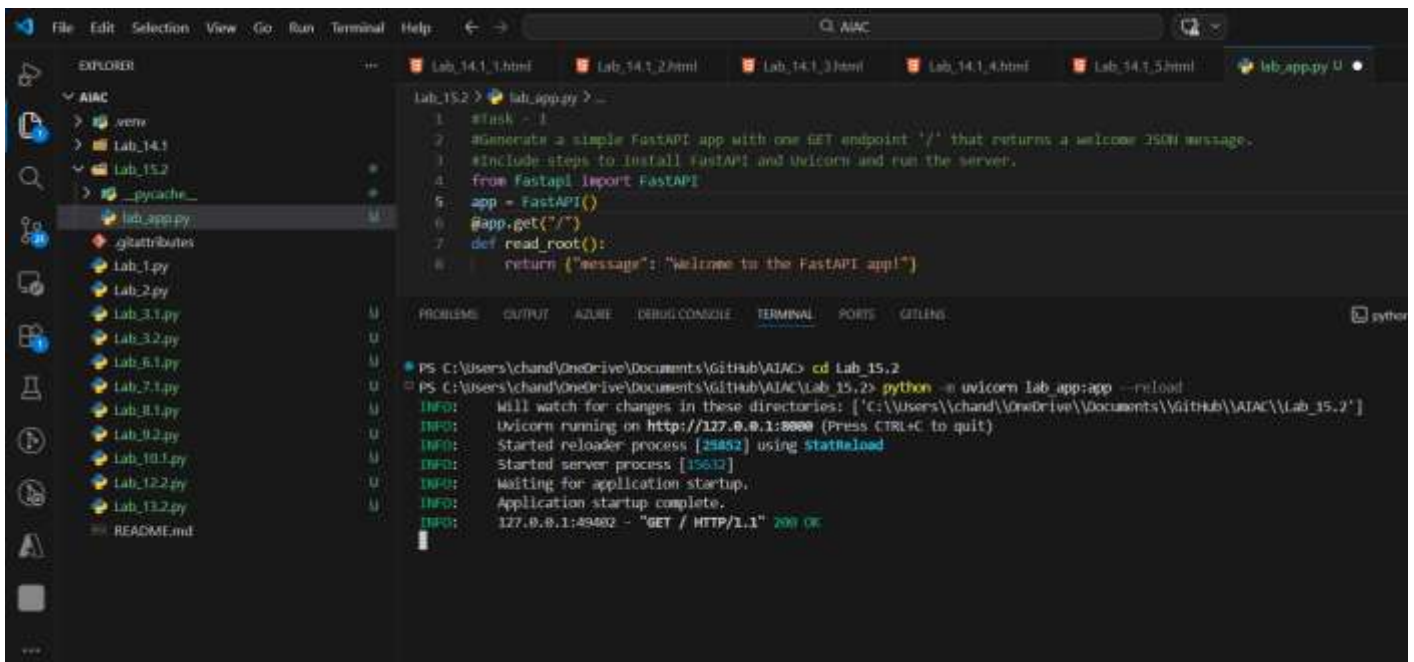
Expected Output:

Running the server and accessing:

`http://127.0.0.1:8000/`

Response:

```
{"message": "Welcome to AI-powered FastAPI"}
```

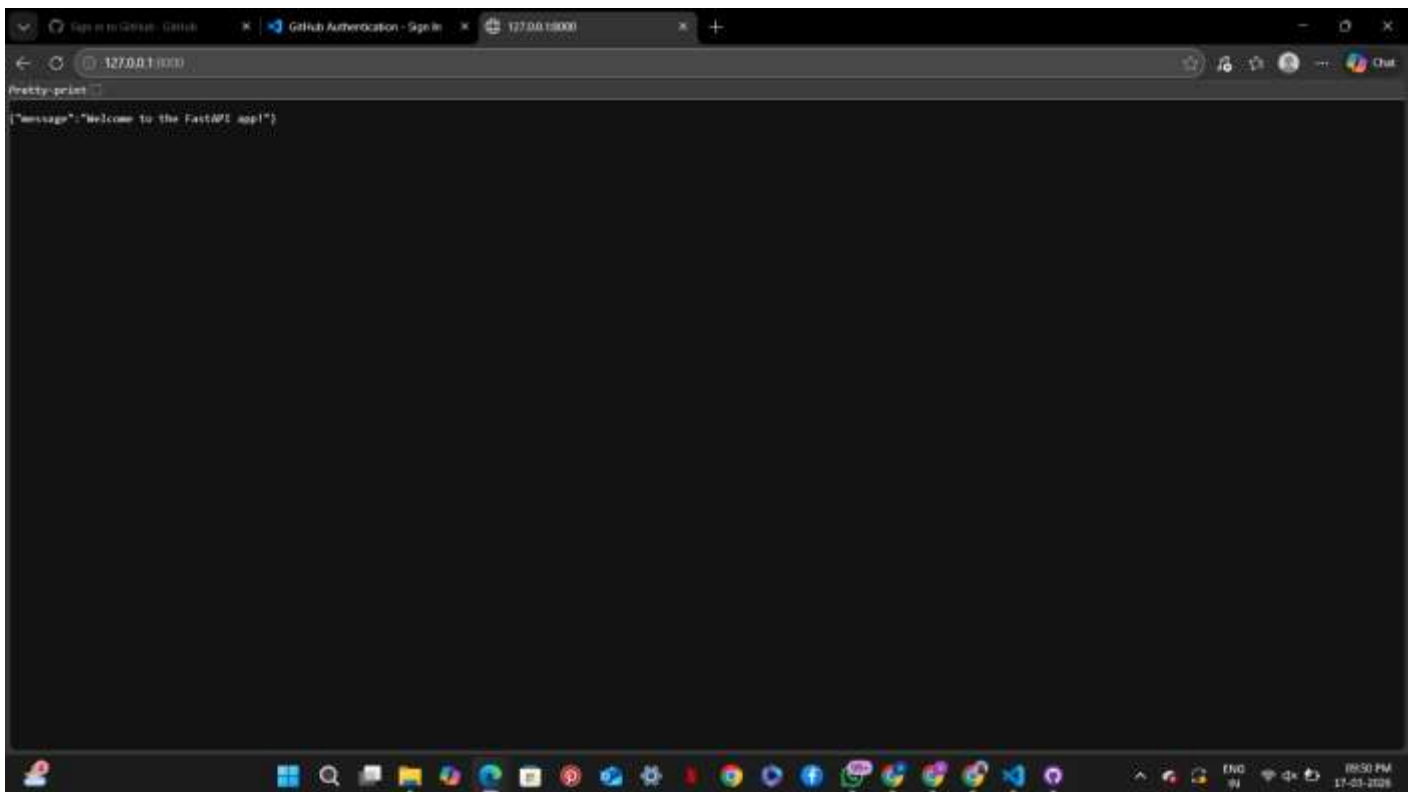


The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'AIAC' with a file 'lab_app.py' selected. The main editor displays the code for 'lab_app.py', which is a simple FastAPI application with a single GET endpoint. The code is as follows:

```
1 #Task - 1
2 #Generate a simple FastAPI app with one GET endpoint '/' that returns a welcome JSON message.
3 #Include steps to install FastAPI and Uvicorn and run the server.
4 from fastapi import FastAPI
5 app = FastAPI()
6 @app.get("/")
7 def read_root():
8     return {"message": "Welcome to the FastAPI app!"}
```

The TERMINAL pane at the bottom shows the command prompt and the output of running the application. The command used is `python -m uvicorn lab_app:app --reload`. The output shows that the application is running on `http://127.0.0.1:8000` and that the server process has started. A test request is also shown:

```
PS C:\Users\chand\OneDrive\Documents\Github\AIAC> cd Lab_15.2
PS C:\Users\chand\OneDrive\Documents\Github\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\chand\OneDrive\Documents\Github\AIAC\Lab_15.2']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [29852] using StatReload
INFO: Started server process [15632]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:49482 - "GET / HTTP/1.1" 200 OK
```



ANALYSIS

- Installs FastAPI and Uvicorn for backend development.
- Creates a basic API with one GET endpoint.
- Returns a JSON response to the client.
- Demonstrates how to run a server locally.
- Builds foundation for further API tasks.

Task Descriptor- 2: (API – Create and Read Items)

Instructions:

- Implement endpoints to add and retrieve products.
- Use a Python list as temporary storage.

Sample Code:

```
items = []

@app.get("/items")
def get_items():
    return items

@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Expected Output:

GET /items → []

POST /items with {"name":"Pen","price":20}

→ {"message":"Item created","item":{"name":"Pen","price":20}}

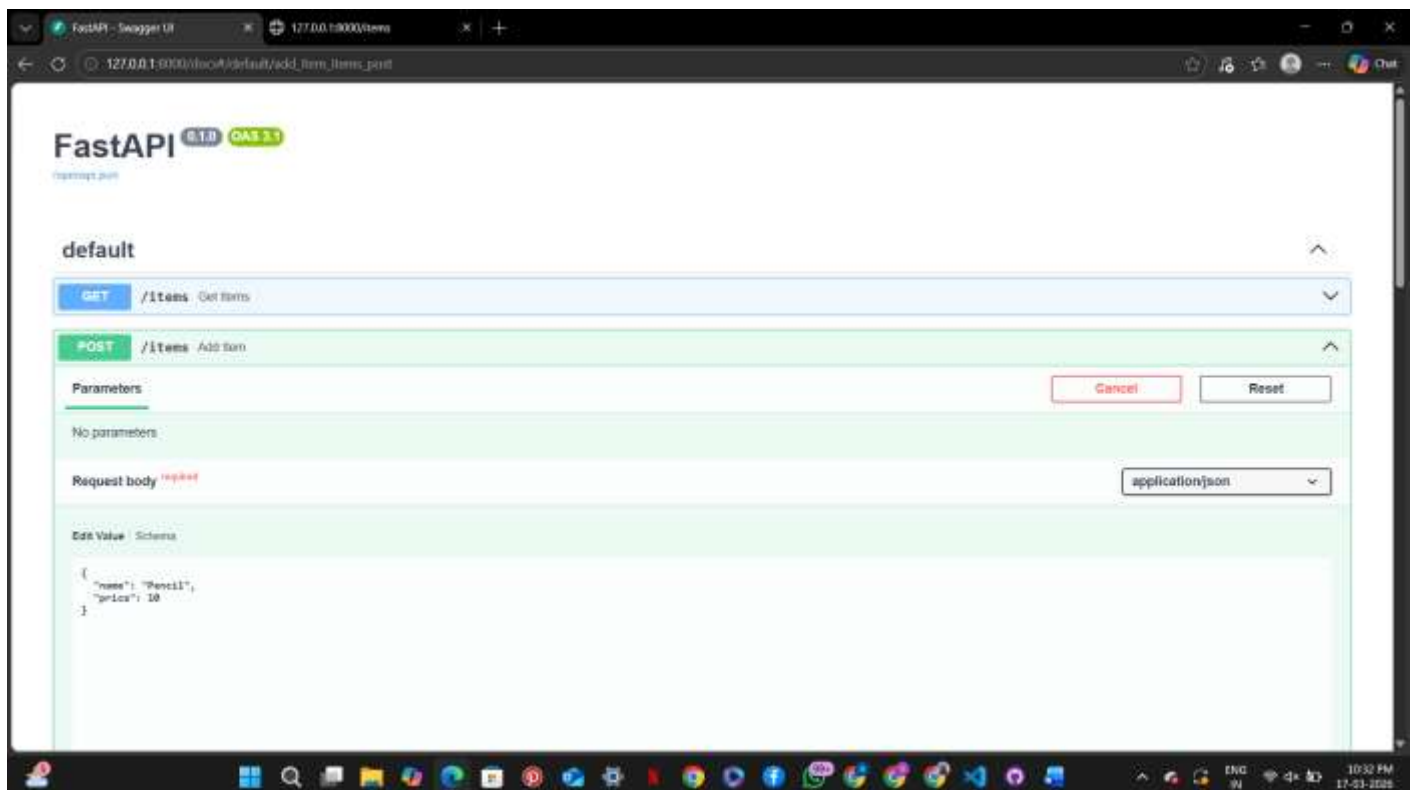
```
Lab_15.2 > lab_app.py > add_item

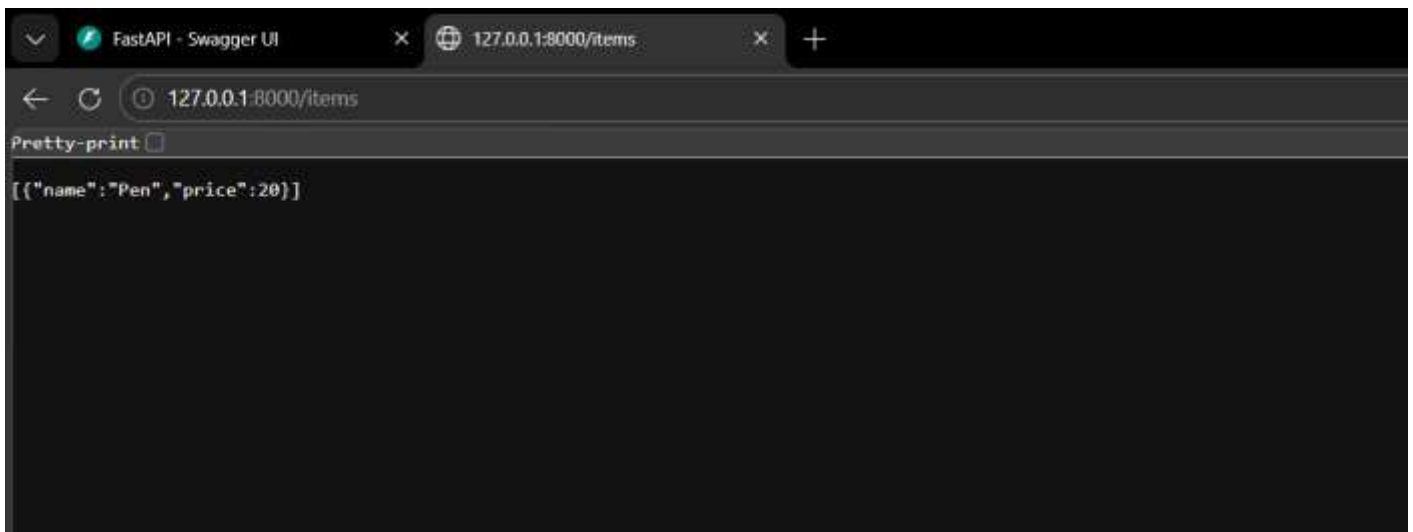
10 #Task - 2
11 #Create FastAPI endpoints to GET and POST items using a Python list as storage. POST should add item, GET should return all items.
12 from fastapi import FastAPI
13 app = FastAPI()
14 items = []
15 @app.get("/items")
16 def get_items():
17     return items
18 @app.post("/items")
19 def add_item(item: dict):
20     items.append(item)
21     return {"message": "Item created", "item": item}
```

PROBLEMS OUTPUT AZURE DEBUG CONSOLE TERMINAL PORTS GITLENS

python - Lab_15.2 + -

```
PS C:\Users\chand\OneDrive\Documents\Github\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO: Stopping reloader process [1708]
PS C:\Users\chand\OneDrive\Documents\Github\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\chand\\OneDrive\\Documents\\Github\\AIAC\\Lab_15.2']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [11760] using StatReload
INFO: Started server process [5064]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:53743 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:60648 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:60648 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:56450 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:63568 - "POST /items HTTP/1.1" 200 OK
INFO: 127.0.0.1:51910 - "GET /items HTTP/1.1" 200 OK
```





ANALYSIS

- Implements GET and POST methods.
- Uses a Python list as temporary storage.
- Allows adding new items dynamically.
- Retrieves all stored items.
- Introduces basic CRUD operations.

Task Descriptor- 3: (Update Existing Item)

Instructions:

- Use AI to generate an endpoint to update an item using its index.

Sample Code:

```
@app.put("/items/{index}")  
  
def update_item(index: int, item: dict):  
    if index >= len(items):  
        return {"error": "Item not found"}  
    items[index] = item  
    return {"message": "Item updated", "item": item}
```

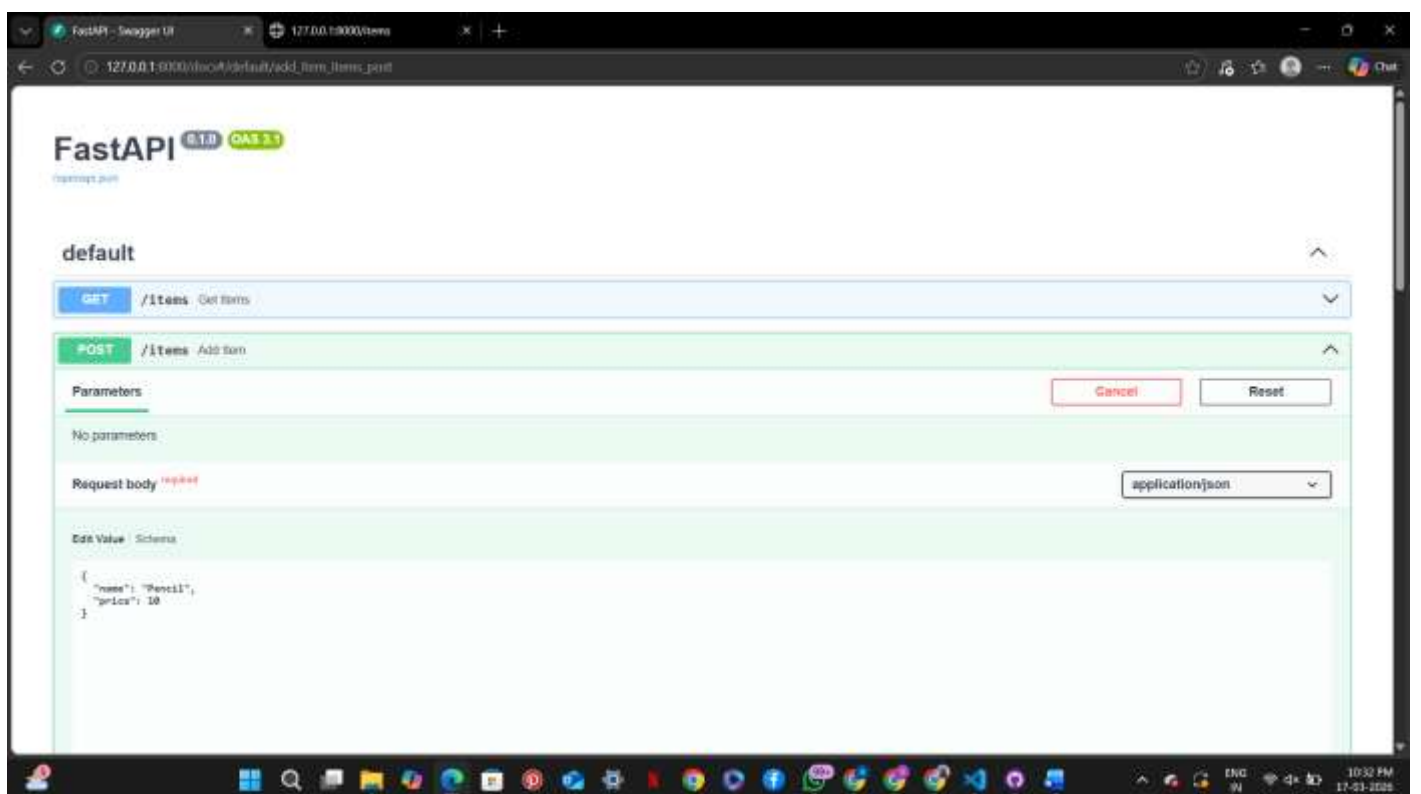
Expected Output:

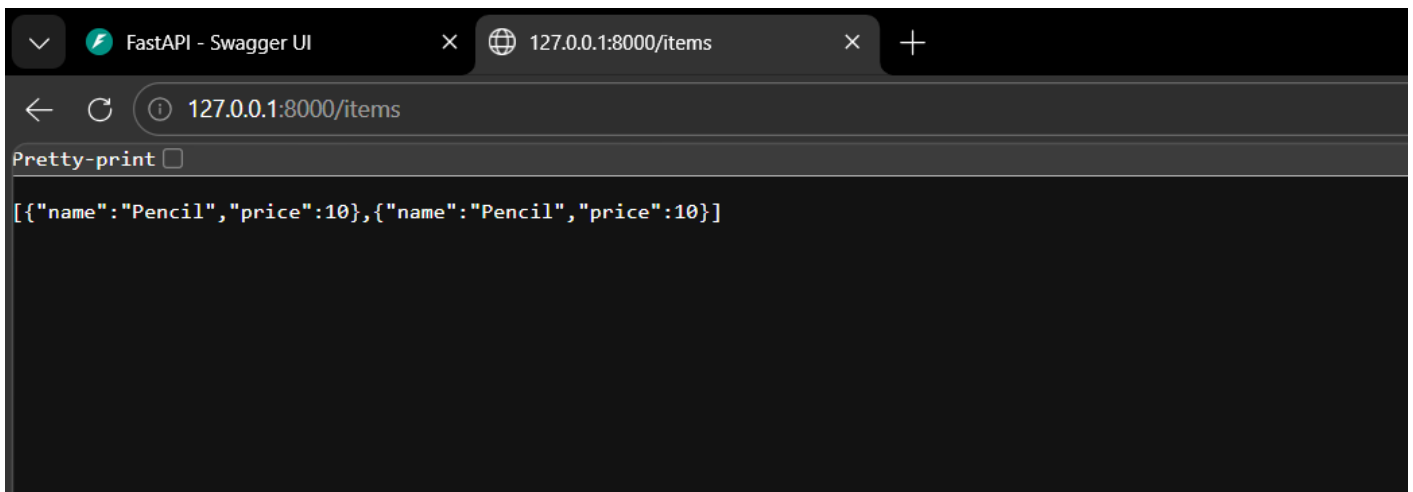
```
PUT /items/0 with {"name": "Pencil", "price": 10}  
→ {"message": "Item updated", "item": {"name": "Pencil", "price": 10}}
```

```
Lab_15.2 > lab_app.py > ...
23 #Task - 3
24 #Create a FastAPI PUT endpoint to update an item by index in a list. Handle case when item is not found.
25 from fastapi import FastAPI, HTTPException
26 app = FastAPI()
27 items = []
28 @app.get("/items")
29 def get_items():
30     return items
31 @app.post("/items")
32 def add_item(item: dict):
33     items.append(item)
34     return {"message": "Item created", "item": item}
35 @app.put("/items/{index}")
36 def update_item(index: int, item: dict):
37     if index < 0 or index >= len(items):
38         raise HTTPException(status_code=404, detail="Item not found")
39     items[index] = item
40     return {"message": "Item updated", "item": item}
41
```

PROBLEMS OUTPUT AZURE DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\chand\OneDrive\Documents\Github\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
PS C:\Users\chand\OneDrive\Documents\Github\AIAC\Lab_15.2> python -m uvicorn lab_app:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\chand\\OneDrive\\Documents\\Github\\AIAC\\Lab_15.2']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [24780] using StatReload
INFO: Started server process [24312]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:57721 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:58416 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:58416 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:51492 - "POST /items HTTP/1.1" 200 OK
INFO: 127.0.0.1:51492 - "POST /items HTTP/1.1" 200 OK
INFO: 127.0.0.1:64316 - "GET /items HTTP/1.1" 200 OK
```





ANALYSIS

- Uses PUT method to update existing data.
- Updates item based on index position.
- Checks for invalid index (error handling).
- Replaces old data with new data.
- Improves data manipulation skills.

Task Descriptor- 4: (Delete Item)

Instructions:

- Implement deletion of an item by index.

Sample Code:

```
@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Expected Output:

DELETE /items/0

→ {"message": "Item removed", "item": {"name": "Pencil", "price": 10}}

- Deletes item using index value.
- Validates index before deletion.
- Returns removed item as response.
- Ensures safe data handling.

Task Descriptor -5: (Add Auto-Generated Documentation)

Instructions:

- Use AI to add inline comments and docstrings for all endpoints.
- Optionally, integrate Swagger / Flask-RESTX to auto-generate API documentation.

Sample Code:

```
@app.route('/items', methods=['GET'])
```

```
def get_items():
```

```
    """
```

```
    GET /items
```

```
    Returns a list of all items in the store.
```

```
    """
```

```
    return jsonify(items)
```

Expected Output:

- Clear documentation for all endpoints, e.g., via /docs if using

Swagger.

- Students should be able to see endpoint description, methods, and sample payloads.

The image shows a development environment with a code editor and a web browser. The code editor displays the source code for a FastAPI application named `lab_app.py`. The code includes imports for `FastAPI` and `HTTPException`, and defines several endpoints: `get_items`, `add_item`, `update_item`, and `delete_item`. The `get_items` endpoint returns a list of items. The `add_item` endpoint adds a new item to the list. The `update_item` endpoint updates an existing item. The `delete_item` endpoint removes an item from the list. The code also includes docstrings and comments to explain the endpoints and enable auto-generated Swagger docs.

The terminal window shows the command `python -m uvicorn lab_app:app --reload` being executed, and the output indicates that the application is running successfully.

The web browser displays the Swagger UI for the FastAPI application. The UI shows the default endpoints: `GET /items` (Get items), `POST /items` (Add item), `PUT /items/{index}` (Update item), and `DELETE /items/{index}` (Delete item). The UI also displays the schemas for the endpoints, including `HTTPValidationError` and `ValidationError`.

ANALYSIS

- Adds docstrings to explain endpoints.
- Enables automatic Swagger documentation.
- Helps users understand API usage easily.
- Displays request/response formats.
- Improves readability and usability of API.